

Partie 2 : Application mobile front-end



1. Contexte et enjeux de la mobilité terrain

Si l'infrastructure back-end de LogiChain assure la centralisation et la sécurité des données, sa valeur ajoutée repose entièrement sur sa capacité à être alimentée en temps réel par les opérateurs sur le terrain. L'organisation d'événements implique des zones blanches (absence de couverture 4G/5G), des conditions météorologiques variables et des opérateurs travaillant dans l'urgence.

L'application mobile n'est donc pas une simple vitrine d'affichage, mais un outil industriel critique. Elle doit masquer la complexité de l'architecture distribuée sous une ergonomie sans faille, tout en garantissant la capture, le stockage local et la synchronisation asynchrone des données logistiques sans bloquer l'utilisateur.

2. Objectifs du projet

- Fournir une interface utilisateur (UI) performante et résiliente, capable de fonctionner de manière transparente lors des transitions entre les zones connectées et déconnectées.
- Exploiter nativement les capacités matérielles du terminal (géolocalisation haute précision, caméra pour le scan intensif de QR codes/Code-barres) avec un minimum de latence.
- Assurer l'intégrité des données à la source avant synchronisation, en appliquant des règles de validation strictes directement sur le client.
- Gérer intelligemment la charge réseau lors du retour de connexion via des stratégies de réconciliation de données (synchronisation en arrière-plan).

3. Périmètre technique global

L'application doit être développée selon les standards de l'industrie mobile moderne, en s'appuyant sur un socle technique solide :

- **Framework mobile** : React Native
- **Stockage local (Offline-First)** : Implémentation d'une base de données locale embarquée (SQLite) capable de gérer des milliers d'enregistrements (Events, Items) et d'exécuter des requêtes relationnelles ou spatiales sur le terminal.
- **Communication réseau** : Consommation de l'API REST LogiChain via des requêtes HTTP sécurisées (gestion des tokens JWT, intercepteurs pour le rafraîchissement des sessions).
- **Temps réel** : Intégration de clients WebSockets ou écoute des Server-Sent Events (SSE) pour la réception d'alertes critiques push.

4. Fonctionnalités clés par profil utilisateur

Administrateurs & Responsables logistiques

- Configuration globale de l'événement et découpage cartographique des zones (coordonnées géospatiales).
- Suivi des indicateurs clés (KPI) via un tableau de bord d'agrégation : état des stocks, calcul de l'empreinte carbone consolidée en temps réel, détection des goulots d'étranglement.
- Supervision des transferts de responsabilité et validation des feuilles de route des transporteurs.

Agents de terrain & Prestataires (Interface Mobile)

- **Authentification et onboarding** : Connexion sécurisée et téléchargement initial du référentiel de données (mise en cache du secteur assigné).
- **Dashboard opérationnel** : Visualisation immédiate des tâches prioritaires, des itinéraires (intégration cartographique des données GeoJSON) et des alertes.
- **Module de scan industriel** : Interface optimisée pour la lecture rapide et en rafale d'identifiants matériels, avec retour haptique et sonore pour valider l'action sans regarder l'écran.
- **Mode dégradé transparent** : Possibilité de valider des étapes logistiques et de déclarer des anomalies géolocalisées même hors réseau. Le système doit empiler ces actions dans une file d'attente locale (queue).
- **Centre de synchronisation** : Interface technique permettant à l'utilisateur de forcer la synchronisation, de visualiser l'état de la file d'attente des requêtes sortantes, et d'être notifié des conflits éventuels (mécanisme de verrouillage optimiste côté API).

5. Contraintes et exigences non-fonctionnelles

- **Architecture logicielle rigoureuse** : Tout comme pour l'API, l'approche "vibe coding" ou le code spaghetti mêlant logique métier, appels réseau et rendu visuel au sein du même composant est strictement interdite. Vous devez appliquer des principes de conception clairs.
 - *Views/Components* : Uniquement dédiés au rendu et à la capture des événements utilisateurs.
 - *Services/Hooks* : Encapsulation de la logique métier et des accès aux données (API ou DB locale).
- **Optimistic UI** : L'interface doit refléter immédiatement le succès d'une action de l'utilisateur (ex: marquer un équipement comme "livré") sans attendre la réponse du serveur, tout en prévoyant un mécanisme de "rollback" visuel en cas d'échec définitif de la synchronisation.
- **Typage fort de bout en bout** : Les interfaces côté Front-End doivent correspondre rigoureusement aux schémas JSON et aux DTOs exposés par l'API Node.js.
- **Gestion des ressources (Battery & Memory)** : La localisation en arrière-plan et le polling réseau doivent être gérés avec parcimonie pour éviter la décharge rapide des terminaux professionnels.